

1 Introducción

1.1 Visión del Proyecto

Sazón Caribeño es un Sistema Integral de Gestión para Restaurantes desarrollado con el propósito de modernizar y optimizar la administración operativa de un establecimiento gastronómico mediante la digitalización de sus procesos críticos.

La necesidad de este sistema surge a partir de los desafíos cotidianos que enfrentan muchos restaurantes al administrar manualmente sus ventas, inventario, personal y recursos financieros. La utilización de registros físicos o herramientas aisladas dificulta el control de la información, incrementa la posibilidad de errores humanos y limita la capacidad de tomar decisiones oportunas basadas en datos confiables.

Con el fin de atender estas necesidades, Sazón Caribeño integra en una única plataforma los principales procesos administrativos y operativos del restaurante, permitiendo controlar el inventario de insumos, administrar el menú, registrar ventas mediante un punto de venta (POS), supervisar la asistencia del personal, gestionar gastos operativos, calcular la nómina y generar reportes financieros de manera centralizada.

La arquitectura del sistema ha sido concebida bajo principios de modularidad, mantenibilidad y escalabilidad, utilizando una API REST como núcleo de comunicación entre el backend y el frontend. Esta decisión permite que la plataforma pueda evolucionar en el futuro incorporando nuevas funcionalidades, integraciones con servicios externos y aplicaciones móviles sin afectar la estructura principal del sistema.

Más allá de automatizar tareas, la visión de Sazón Caribeño es convertirse en una herramienta de apoyo para la gestión empresarial del restaurante, proporcionando información precisa, reduciendo tiempos operativos, fortaleciendo el control interno y facilitando la toma de decisiones basada en datos.

El presente documento constituye la especificación funcional y técnica del sistema, describiendo la arquitectura, los requisitos, las reglas de negocio y las decisiones de diseño que respaldan su desarrollo y mantenimiento.

1.2 Objetivo General

Desarrollar un Sistema Integral de Gestión para Restaurantes que permita administrar de forma centralizada los procesos operativos y administrativos de Sazón Caribeño, integrando la gestión de usuarios, personal, inventario, menú, punto de venta, gastos, nómina y reportes financieros en una plataforma web segura, eficiente y escalable, con el propósito de optimizar la operación del restaurante, fortalecer el control interno y proporcionar información confiable para la toma de decisiones.

1.3 Objetivos Específicos

Para alcanzar el objetivo general, el sistema deberá cumplir los siguientes objetivos específicos:

- Automatizar el proceso de registro de ventas mediante un Punto de Venta (POS) intuitivo y eficiente.
- Mantener un control preciso del inventario de productos e insumos, actualizando automáticamente las existencias conforme a las ventas realizadas y a los ajustes de inventario.
- Gestionar la administración de platillos y recetas, permitiendo definir los ingredientes y cantidades necesarias para su preparación.
- Administrar la información del personal, incluyendo usuarios, puestos de trabajo, salarios y control de asistencia.
- Implementar un sistema de control de turnos basado en validación de red y verificación periódica (Heartbeat) para garantizar la autenticidad de los registros de asistencia.
- Registrar y controlar los gastos operativos del restaurante, permitiendo su integración con los reportes financieros.
- Calcular automáticamente la nómina del personal considerando jornadas laborales, horas extraordinarias y salario base.
- Generar reportes financieros que faciliten el análisis de ingresos, egresos y utilidad del negocio.
- Implementar un modelo de seguridad basado en roles que garantice el acceso únicamente a las funciones autorizadas para cada tipo de usuario.
- Diseñar una arquitectura modular y escalable que facilite el mantenimiento del sistema y la incorporación de nuevas funcionalidades en futuras versiones.

• 1.5 Público Objetivo

El sistema Sazón Caribeño está dirigido principalmente a pequeños y medianos establecimientos del sector gastronómico que requieren una herramienta tecnológica para optimizar el control de sus procesos administrativos y operativos.

Dentro de la organización, la aplicación está orientada a los siguientes perfiles de usuario:

Administrador

Responsable de la administración general del sistema. Tiene acceso completo a las funcionalidades de configuración, gestión de usuarios, mantenimiento de la información y supervisión de la seguridad de la plataforma.

Gerente

Responsable de la supervisión operativa del restaurante. Gestiona el inventario, administra el personal, registra gastos, consulta reportes financieros, supervisa la nómina y controla el funcionamiento general del establecimiento.

Vendedor

Responsable del proceso de atención al cliente mediante el Punto de Venta (POS). Registra ventas, administra su turno de trabajo y opera bajo las restricciones de seguridad definidas por el sistema para garantizar la integridad de las operaciones comerciales.

1.7 Tecnologías Utilizadas

El desarrollo del sistema Sazón Caribeño se fundamenta en tecnologías modernas orientadas a la construcción de aplicaciones web escalables, mantenibles y de alto rendimiento.

Tecnología	Versión	Descripción
Python	3.14	Lenguaje principal utilizado para el desarrollo del backend.
FastAPI	0.138	Framework para el desarrollo de la API REST del sistema.
SQLAlchemy	2.x	ORM utilizado para el mapeo objeto-relacional y acceso a la base de datos.
Alembic	1.18	Herramienta para el control de versiones y migraciones de la base de datos.
MySQL	8.x	Sistema gestor de base de datos relacional.

Tecnología	Versión	Descripción
PyMySQL	1.2	Controlador de conexión entre Python y MySQL.
Pydantic	2.x	Validación y serialización de datos utilizados por la API.
HTML5	-	Estructura de las interfaces de usuario.
Tailwind CSS	4.x	Framework CSS utilizado para el diseño de la interfaz gráfica.
JavaScript (ES6+)	-	Implementación de la lógica del cliente y comunicación con la API mediante Fetch API.
Material Symbols	-	Biblioteca de iconografía utilizada en la interfaz del sistema.
Git	-	Sistema de control de versiones utilizado durante el desarrollo del proyecto.

2. Análisis del Negocio

El desarrollo de un sistema de información debe responder a las necesidades reales de la organización para la cual ha sido concebido. Antes de definir la arquitectura, los módulos o las tecnologías a utilizar, es fundamental comprender el contexto operativo del restaurante, identificar las problemáticas existentes y establecer de qué manera la solución tecnológica contribuirá al cumplimiento de los objetivos del negocio.

En este capítulo se describe el entorno en el que opera Sazón Caribeño, los desafíos administrativos identificados durante el análisis de requerimientos y la propuesta de solución planteada mediante el desarrollo del Sistema Integral de Gestión para Restaurantes.

2.1 Descripción del Restaurante

Sazón Caribeño es un restaurante dedicado a la preparación y comercialización de alimentos y bebidas, cuyo funcionamiento diario involucra procesos administrativos y operativos que requieren un control constante sobre los recursos del negocio.

Las actividades principales del restaurante incluyen la administración del inventario de materias primas e insumos, la elaboración de platillos a partir de recetas previamente definidas, la atención al cliente mediante un punto de venta (POS), la gestión del personal, el registro de gastos operativos y el seguimiento del desempeño financiero del establecimiento.

Como en muchos negocios gastronómicos de pequeña y mediana escala, las operaciones involucran múltiples áreas que interactúan entre sí. Una venta realizada en el punto de venta impacta directamente el inventario; los movimientos del personal afectan el cálculo de la nómina; y los gastos operativos influyen en la rentabilidad diaria del negocio. Esta relación entre procesos hace necesaria una administración centralizada que garantice la consistencia de la información.

Debido a la naturaleza dinámica del restaurante, resulta indispensable disponer de información actualizada en tiempo real que permita supervisar las operaciones, reducir errores administrativos y facilitar la toma de decisiones basada en datos confiables.

2.2 Problema Identificado

Durante el análisis del negocio se identificó la necesidad de mejorar el control y la integración de los procesos administrativos del restaurante.

La gestión manual o parcialmente digitalizada de las ventas, el inventario, los gastos y la información del personal incrementa el riesgo de inconsistencias en los registros, dificulta el seguimiento de las operaciones diarias y limita la disponibilidad de información precisa para la toma de decisiones.

Entre las principales problemáticas identificadas se encuentran:

- Falta de integración entre los procesos de ventas e inventario.
- Riesgo de diferencias entre el inventario físico y el inventario registrado.
- Dificultad para conocer la rentabilidad diaria del restaurante.
- Ausencia de un mecanismo automatizado para el control de asistencia del personal.
- Procesos administrativos que requieren cálculos manuales, aumentando la probabilidad de errores.
- Limitado acceso a indicadores que apoyen la planificación y supervisión de las operaciones.
- Necesidad de restringir el acceso a determinadas funciones del sistema según las responsabilidades de cada usuario.

Estas situaciones afectan la eficiencia operativa del restaurante y pueden generar pérdidas económicas, retrasos administrativos y dificultades para mantener un control adecuado de los recursos del negocio.

2.3 Solución Propuesta

Como respuesta a las necesidades identificadas, se propone el desarrollo del Sistema Integral de Gestión para Restaurantes Sazón Caribeño, una plataforma web diseñada para centralizar la administración de los principales procesos operativos y administrativos del establecimiento.

La solución integra en un único sistema los módulos de gestión de usuarios, administración del personal, control de turnos, inventario, menú de platillos, punto de venta (POS), gestión de gastos, cálculo de nómina y generación de reportes financieros, permitiendo que la información fluya de manera consistente entre todas las áreas del negocio.

La arquitectura del sistema está basada en una API REST desarrollada con FastAPI y un frontend independiente, favoreciendo la modularidad, el mantenimiento y la posibilidad de incorporar nuevas funcionalidades en futuras versiones.

Asimismo, el sistema implementa mecanismos de seguridad basados en control de acceso por roles, validación de turnos y verificación periódica de actividad mediante Heartbeat, contribuyendo a proteger la integridad de la información y garantizar el correcto uso de la plataforma.

Esta solución permitirá sustituir procesos manuales por procedimientos automatizados, reduciendo tiempos operativos, mejorando la calidad de los registros y proporcionando información confiable para la gestión del restaurante.

2.4 Beneficios Esperados

La implementación del Sistema Integral de Gestión para Restaurantes Sazón Caribeño permitirá obtener beneficios tanto operativos como administrativos, contribuyendo a mejorar la eficiencia y el control del negocio.

Entre los principales beneficios esperados se encuentran:

- Centralización de la información administrativa y operativa en una única plataforma.
- Reducción de errores derivados de procesos manuales.
- Actualización automática del inventario conforme a las ventas realizadas.
- Mayor precisión en el cálculo de la nómina del personal.
- Control eficiente de los gastos operativos y su impacto en la rentabilidad del restaurante.
- Disponibilidad de reportes financieros que apoyen la toma de decisiones estratégicas.
- Fortalecimiento de la seguridad mediante un modelo de control de acceso basado en roles y mecanismos de validación de turnos.
- Optimización del tiempo requerido para realizar tareas administrativas.
- Mayor trazabilidad de las operaciones realizadas dentro del sistema.
- Base tecnológica preparada para futuras ampliaciones, como soporte para múltiples sucursales, integración con servicios externos y aplicaciones móviles.

En conjunto, estos beneficios contribuirán a incrementar la productividad del restaurante, mejorar el control de los recursos y proporcionar una herramienta confiable para la administración del negocio.

3. Arquitectura del Sistema

La arquitectura de Sazón Caribeño ha sido diseñada bajo un modelo cliente-servidor desacoplado, en el cual el frontend y el backend funcionan como componentes independientes que se comunican mediante una API REST.

Este enfoque permite separar claramente la lógica de negocio de la interfaz de usuario, facilitando el mantenimiento, la escalabilidad y la incorporación de nuevas plataformas de acceso, como aplicaciones móviles o sistemas externos.

La arquitectura está compuesta por cuatro capas principales:

- **Capa de Presentación (Frontend):** Responsable de la interacción con el usuario mediante interfaces web desarrolladas con HTML5, Tailwind CSS y JavaScript.
- **Capa de Servicios (API REST):** Implementada mediante FastAPI, centraliza la lógica de negocio, valida las solicitudes recibidas, aplica las reglas del sistema y controla el acceso a los recursos.
- **Capa de Persistencia:** Utiliza SQLAlchemy como ORM para gestionar la comunicación entre la aplicación y la base de datos relacional.
- **Capa de Datos:** Constituida por una base de datos MySQL que almacena de forma estructurada la información del sistema, garantizando la integridad y consistencia de los datos.

Esta arquitectura permite mantener un bajo acoplamiento entre los componentes del sistema, favoreciendo la reutilización del código y simplificando futuras ampliaciones del proyecto.

3.1 Arquitectura General

El flujo general de funcionamiento del sistema puede resumirse de la siguiente manera:

1. El usuario accede a la aplicación mediante un navegador web.
2. El frontend procesa la interacción del usuario y envía solicitudes HTTP a la API REST.
3. FastAPI valida la autenticación, los permisos del usuario y las reglas de negocio correspondientes.
4. SQLAlchemy gestiona la interacción con la base de datos mediante modelos de dominio.
5. La información procesada es devuelta al frontend en formato JSON.
6. La interfaz actualiza la información mostrada al usuario sin necesidad de recargar completamente la página.

Este modelo favorece la modularidad de la aplicación y facilita la integración con otros sistemas en futuras versiones.

3.2 Componentes Principales

La solución se encuentra organizada en los siguientes componentes principales:

- Frontend Web.
- API REST.
- Sistema de Autenticación.
- Control de Acceso Basado en Roles (RBAC).
- Gestión de Usuarios.
- Gestión de Personal.
- Gestión de Turnos.
- Gestión de Inventario.
- Gestión del Menú y Recetas.
- Punto de Venta (POS).
- Gestión de Gastos.

- Gestión de Nómina.
- Reportes Financieros.
- Servicio de Auditoría.
- Base de Datos Relacional.

Cada componente posee responsabilidades claramente definidas y se comunica con los demás únicamente mediante las interfaces establecidas por la arquitectura del sistema, reduciendo el acoplamiento entre módulos.

4. Requerimientos Funcionales

Los requerimientos funcionales describen las capacidades que el sistema deberá proporcionar para satisfacer las necesidades operativas del restaurante. Cada requerimiento representa una funcionalidad observable por el usuario y constituye la base para el desarrollo, las pruebas y la validación del sistema.

Los requerimientos se agrupan según el módulo funcional al que pertenecen.

4.1 Gestión de Usuarios

RF-001. El sistema deberá permitir el registro de nuevos usuarios autorizados.

RF-002. El sistema deberá asociar cada usuario a un empleado del restaurante.

RF-003. El sistema deberá asignar un rol de acceso a cada usuario.

RF-004. El sistema deberá permitir el restablecimiento de contraseñas por parte del administrador.

RF-005. El sistema deberá permitir consultar la información del usuario autenticado.

4.2 Gestión del Personal

RF-006. El sistema deberá registrar la información laboral de cada empleado.

RF-007. El sistema deberá administrar los puestos de trabajo desempeñados por los empleados.

RF-008. El sistema deberá almacenar el salario base correspondiente a cada puesto.

RF-009. El sistema deberá calcular automáticamente la nómina quincenal.

RF-010. El sistema deberá calcular las horas extraordinarias cuando la jornada laboral exceda las ocho horas diarias.

4.3 Gestión de Turnos

RF-011. El sistema deberá permitir iniciar un turno laboral.

RF-012. El sistema deberá validar las condiciones de seguridad definidas para el inicio del turno.

RF-013. El sistema deberá registrar periódicamente la actividad del turno mediante el mecanismo Heartbeat.

RF-014. El sistema deberá permitir finalizar un turno de forma manual o automática.

RF-015. El sistema deberá conservar el historial de turnos realizados por cada empleado.

(...continuará para Inventario, Platillos, POS, Gastos, Reportes, Auditoría y Configuración del Sistema.)

5. Requerimientos No Funcionales

Los requerimientos no funcionales establecen las características de calidad que deberá cumplir el sistema durante su operación. Estos requisitos garantizan aspectos relacionados con la seguridad, el rendimiento, la disponibilidad, la mantenibilidad y la experiencia del usuario.

Seguridad

- El sistema deberá implementar un modelo de control de acceso basado en roles (RBAC).
- Todas las operaciones protegidas deberán validar la identidad del usuario autenticado.
- Las contraseñas deberán almacenarse utilizando algoritmos de hash seguros.
- Todas las operaciones críticas deberán registrarse mediante el Servicio de Auditoría.
- Los vendedores únicamente podrán operar el Punto de Venta cuando posean un turno activo.

Rendimiento

- El tiempo promedio de respuesta de la API no deberá superar los dos segundos para operaciones comunes.
- El sistema deberá soportar múltiples solicitudes concurrentes sin degradar significativamente el rendimiento.

Disponibilidad

- El sistema deberá encontrarse disponible durante el horario operativo del restaurante.
- La información deberá almacenarse de forma consistente incluso ante cierres inesperados de la aplicación.

Escalabilidad

- La arquitectura deberá permitir incorporar nuevos módulos sin afectar el funcionamiento de los existentes.
- El sistema deberá facilitar futuras integraciones con aplicaciones móviles, múltiples sucursales y servicios externos.

Mantenibilidad

- El código deberá organizarse de manera modular.
- La API deberá mantener una estructura consistente para facilitar su mantenimiento.
- La base de datos deberá administrarse mediante migraciones versionadas.

Usabilidad

- La interfaz deberá ser intuitiva y consistente en todos los módulos.
- El sistema deberá adaptarse correctamente a diferentes tamaños de pantalla.
- Los mensajes mostrados al usuario deberán ser claros y comprensibles.

Integridad de la Información

- Todas las operaciones que modifiquen datos deberán preservar la consistencia de la base de datos.
- Los movimientos de inventario deberán ser completamente trazables.
- El sistema deberá registrar un historial de auditoría de las operaciones administrativas realizadas por los usuarios autorizados.

6. Reglas de Negocio

Las reglas de negocio representan las políticas, restricciones y condiciones que gobiernan el funcionamiento del sistema. Estas reglas reflejan la forma en que opera el restaurante y garantizan que todas las transacciones realizadas respeten los procedimientos establecidos por la organización.

Las reglas aquí descritas constituyen la base de la lógica implementada en la aplicación y deberán mantenerse consistentes durante todo el ciclo de vida del sistema.

6.1 Reglas de Seguridad y Control de Acceso

RN-101.

Todo usuario deberá autenticarse antes de acceder a cualquier funcionalidad protegida del sistema.

RN-102.

Cada usuario tendrá asignado un único rol de acceso.

RN-103.

Los permisos del sistema estarán determinados exclusivamente por el rol del usuario y no por su puesto laboral.

RN-104.

Todo empleado podrá desempeñar cualquier puesto definido por el restaurante sin que ello implique un cambio automático en sus permisos dentro del sistema.

RN-105.

Las operaciones restringidas deberán validar el rol del usuario antes de ejecutarse.

RN-106.

Toda acción administrativa deberá registrarse en el historial de auditoría.

RN-107.

Las contraseñas nunca serán almacenadas en texto plano.

RN-108.

Las sesiones deberán invalidarse cuando el usuario cierre sesión o pierda autorización para continuar operando.

6.2 Reglas para la Gestión del Personal

RN-201.

Todo empleado deberá estar asociado a un usuario del sistema.

RN-202.

Cada empleado tendrá asignado un puesto laboral.

RN-203.

El puesto laboral representa la función desempeñada dentro del restaurante y es independiente del rol utilizado para acceder al sistema.

RN-204.

El salario base será definido por el puesto laboral correspondiente.

RN-205.

El sistema deberá conservar el historial de empleados aun cuando estos dejen de laborar en la empresa.

6.3 Reglas para la Gestión del Inventario**RN-301.**

Todo producto deberá pertenecer a una categoría.

RN-302.

Todo producto deberá estar asociado a una unidad de medida.

RN-303.

El inventario nunca podrá presentar existencias negativas.

RN-304.

Toda modificación manual del inventario deberá quedar registrada en el historial de auditoría.

RN-305.

Los retiros de inventario deberán generar automáticamente un registro de gasto.

RN-306.

El sistema notificará cuando las existencias sean inferiores al stock mínimo configurado.

6.4 Reglas para la Gestión del Menú**RN-401.**

Todo platillo deberá poseer una receta válida.

RN-402.

Una receta estará conformada por uno o más ingredientes.

RN-403.

Cada ingrediente deberá indicar la cantidad necesaria para preparar el platillo.

RN-404.

Un producto podrá formar parte de múltiples recetas.

6.5 Reglas para el Punto de Venta**RN-501.**

Toda venta deberá estar asociada al usuario que la realizó.

RN-502.

Los vendedores únicamente podrán registrar ventas cuando posean un turno activo.

RN-503.

Al confirmar una venta, el sistema descontará automáticamente del inventario todos los ingredientes definidos en la receta del platillo.

RN-504.

Una venta no podrá completarse cuando los insumos disponibles sean insuficientes para preparar alguno de los platillos solicitados.

RN-505.

Toda venta deberá quedar registrada para efectos contables y de auditoría.

6.6 Reglas para la Gestión de Turnos**RN-601.**

Los vendedores deberán iniciar un turno antes de utilizar el Punto de Venta.

RN-602.

El sistema validará la ubicación autorizada mediante la dirección IP configurada para el restaurante.

RN-603.

Mientras un turno permanezca activo, el sistema verificará periódicamente la conexión mediante el mecanismo Heartbeat.

RN-604.

Cuando el Heartbeat deje de recibirse dentro del tiempo establecido, el turno será finalizado automáticamente.

RN-605.

El cierre automático utilizará como hora final el último Heartbeat válido recibido.

6.7 Reglas para la Nómina

RN-701.

La nómina será calculada automáticamente a partir de los turnos registrados.

RN-702.

Las horas trabajadas superiores a la jornada ordinaria serán consideradas horas extraordinarias. (las horas extras no se pagaran doble)

RN-703.

El salario correspondiente a cada empleado será calculado utilizando el salario base definido para su puesto laboral.

6.8 Reglas para Reportes

RN-801.

Los reportes financieros deberán calcularse utilizando únicamente información validada por el sistema.

RN-802.

La utilidad neta será calculada como la diferencia entre ingresos y egresos registrados durante el período consultado.

RN-803.

Los reportes deberán conservar la trazabilidad de las operaciones que los originan.

8. Diseño del Backend

El backend del sistema Sazón Caribeño constituye el núcleo de la aplicación, siendo responsable de implementar la lógica de negocio, garantizar la seguridad de las operaciones, administrar el acceso a los datos y coordinar la comunicación entre la interfaz de usuario y la base de datos.

La arquitectura del backend ha sido diseñada siguiendo el principio de separación de responsabilidades (Separation of Concerns), organizando el sistema en componentes especializados que permiten facilitar el mantenimiento, la escalabilidad y la incorporación de nuevas funcionalidades.

Cada componente posee una responsabilidad claramente definida y se comunica con los demás mediante interfaces controladas, evitando dependencias innecesarias entre módulos.

8.1 Arquitectura en Capas

El backend adopta una arquitectura multicapa, donde cada nivel del sistema cumple una función específica dentro del procesamiento de una solicitud.

El flujo general de ejecución se desarrolla de la siguiente manera:

1. El cliente realiza una solicitud HTTP hacia la API REST.
2. FastAPI recibe la solicitud y la dirige al controlador correspondiente.
3. El controlador valida la información recibida y delega el procesamiento al servicio especializado.
4. El servicio aplica las reglas de negocio necesarias para ejecutar la operación solicitada.
5. Cuando es necesario acceder a la información persistente, el servicio utiliza un repositorio encargado de interactuar con la base de datos mediante SQLAlchemy.
6. Finalmente, la información procesada es devuelta al controlador, el cual genera la respuesta correspondiente para el cliente.

Esta organización permite mantener un bajo acoplamiento entre los componentes del sistema y facilita el crecimiento del proyecto conforme se incorporen nuevos módulos.

8.2 Componentes del Backend

La arquitectura del backend se encuentra dividida en los siguientes componentes funcionales:

API REST

Implementada mediante FastAPI, constituye el punto de entrada para todas las solicitudes realizadas por el frontend. Se encarga de exponer los servicios del sistema mediante endpoints HTTP, validar los datos de entrada y generar las respuestas correspondientes.

Controladores

Los controladores representan los endpoints de la aplicación. Su responsabilidad consiste en recibir las solicitudes del cliente, validar los parámetros de entrada, verificar la autenticación del usuario y delegar el procesamiento de la solicitud al servicio correspondiente.

Los controladores no contienen reglas de negocio; únicamente coordinan la comunicación entre el cliente y los servicios del sistema.

Servicios

Los servicios implementan la lógica de negocio del sistema. En esta capa se ejecutan las validaciones funcionales, se aplican las reglas del negocio y se coordinan las operaciones necesarias para completar cada proceso.

Ejemplos de responsabilidades de los servicios incluyen el registro de ventas, el cálculo de la nómina, la actualización automática del inventario, la generación de reportes financieros y el control de turnos.

Repositorios

Los repositorios encapsulan todas las operaciones relacionadas con el acceso a la base de datos. Su función consiste en consultar, insertar, actualizar y eliminar información utilizando SQLAlchemy como mecanismo de persistencia.

Esta separación evita que la lógica del negocio dependa directamente del motor de base de datos.

Modelos

Los modelos representan las entidades persistentes del sistema y establecen las relaciones existentes entre ellas mediante SQLAlchemy.

Cada modelo corresponde a una tabla de la base de datos y define la estructura de la información almacenada.

Esquemas de Datos

Los esquemas implementados mediante Pydantic permiten validar las solicitudes recibidas y controlar la estructura de las respuestas enviadas por la API.

Esta capa garantiza que únicamente se procesen datos válidos y consistentes.

Base de Datos

La información del sistema se almacena en una base de datos relacional MySQL administrada mediante SQLAlchemy y Alembic, permitiendo mantener el control de versiones de la estructura de datos y garantizando la integridad referencial entre las entidades del sistema.

8.3 Flujo de Procesamiento de una Solicitud

El procesamiento de una operación dentro del sistema sigue una secuencia claramente definida.

Cuando un usuario realiza una acción desde la interfaz web, el frontend envía una solicitud HTTP a la API REST.

La API recibe la petición y la dirige al controlador correspondiente. El controlador valida los datos recibidos, verifica la autenticación y los permisos del usuario y delega la operación al servicio especializado.

Posteriormente, el servicio ejecuta las reglas de negocio asociadas al proceso solicitado. Si la operación requiere acceder o modificar información almacenada, el servicio utiliza el repositorio correspondiente, el cual interactúa con la base de datos mediante SQLAlchemy.

Una vez finalizada la operación, el resultado retorna al controlador, donde se construye una respuesta estructurada utilizando los esquemas definidos por Pydantic. Finalmente, la respuesta es enviada al cliente en formato JSON.

app/

|

|— api/

|

|— usuarios.py

| └─ empleados.py

| └─ turnos.py

| └─ inventario.py

| └─ menu.py

| └─ ventas.py

| └─ gastos.py

| └─ reportes.py

| └─ nomina.py

| └─ auditoria.py

|

| └─ services/

| | └─ usuario_service.py

| | └─ empleado_service.py

| | └─ turno_service.py

| | └─ inventario_service.py

| | └─ receta_service.py

| | └─ venta_service.py

| | └─ gasto_service.py

| | └─ nomina_service.py

| | └─ reporte_service.py

| | └─ auditoria_service.py

|

| └─ repositories/

| | └─ usuario_repository.py

| | └─ empleado_repository.py

| | └─ turno_repository.py

```
| |└─ producto_repository.py
| |└─ venta_repository.py
| |└─ gasto_repository.py
| |└─ ...
|
|└─ models/
|
|└─ schemas/
|
|└─ core/
| |└─ database.py
| |└─ security.py
| |└─ config.py
| |└─ exceptions.py
| |└─ dependencies.py
|
|└─ utils/
| |└─ validators.py
| |└─ calculations.py
| |└─ heartbeat.py
| |└─ audit.py
|
|└─ middleware/
|
└─ main.py
```

8.5 Principios Arquitectónicos

El desarrollo del backend se rige por un conjunto de principios arquitectónicos cuyo propósito es garantizar la calidad, mantenibilidad y escalabilidad del sistema a largo plazo.

Separación de Responsabilidades (Separation of Concerns)

Cada componente del sistema deberá tener una única responsabilidad claramente definida, evitando la concentración de múltiples funciones en un mismo módulo.

Arquitectura por Capas

Las solicitudes deberán procesarse siguiendo la secuencia:

Cliente → API → Controlador → Servicio → Repositorio → Base de Datos.

Cada capa únicamente podrá comunicarse con la capa inmediatamente inferior, reduciendo el acoplamiento entre componentes.

Servicios como Centro de la Lógica del Negocio

Toda regla de negocio deberá implementarse dentro de los servicios del sistema.

Los controladores no deberán contener lógica relacionada con cálculos, validaciones funcionales o procesos administrativos; su responsabilidad será exclusivamente recibir solicitudes, validar parámetros básicos y delegar el procesamiento al servicio correspondiente.

Acceso Centralizado a Datos

Toda interacción con la base de datos deberá realizarse mediante repositorios especializados, evitando consultas directas desde los controladores o servicios.

Modularidad

Cada módulo funcional del sistema deberá mantenerse independiente de los demás en la medida de lo posible, favoreciendo la reutilización del código y la incorporación de nuevas funcionalidades sin afectar componentes existentes.

Escalabilidad

La arquitectura deberá permitir la incorporación de nuevos módulos funcionales, servicios externos y clientes adicionales sin requerir modificaciones significativas en la estructura principal del sistema.

Trazabilidad

Toda operación crítica deberá ser registrada mediante el servicio de auditoría, permitiendo conocer quién realizó cada acción, cuándo ocurrió y cuál fue el resultado de la operación.

9. Diseño del Frontend

El frontend de Sazón Caribeño constituye la capa de presentación del sistema y tiene como objetivo proporcionar una interfaz intuitiva, eficiente y consistente para los diferentes perfiles de usuario.

La aplicación ha sido desarrollada como una interfaz web responsiva que se comunica con el backend mediante una API REST, permitiendo actualizar la información de forma dinámica sin necesidad de recargar completamente las páginas.

El diseño prioriza la simplicidad de uso, minimizando la cantidad de acciones necesarias para completar las tareas más frecuentes dentro de la operación diaria del restaurante.

La interfaz mantiene una identidad visual uniforme en todos los módulos mediante el uso de componentes reutilizables, una paleta de colores consistente y criterios homogéneos de navegación.

9.1 Arquitectura del Frontend

El frontend se encuentra organizado en módulos independientes, donde cada página representa una funcionalidad específica del sistema.

La comunicación entre la interfaz y el backend se realiza exclusivamente mediante solicitudes HTTP a la API REST, utilizando un servicio centralizado encargado de gestionar todas las operaciones de comunicación.

Esta organización evita duplicidad de código y facilita el mantenimiento de la aplicación conforme se incorporan nuevas funcionalidades.

Módulo	Responsabilidad
---------------	------------------------

Dashboard	Información general
-----------	---------------------

Inventario	Productos, categorías, unidades
------------	---------------------------------

POS	Ventas
-----	--------

Menú	Platillos
------	-----------

Módulo	Responsabilidad
---------------	------------------------

Gastos	Gastos
--------	--------

Reportes	Reportes
----------	----------

Personal	Empleados
----------	-----------

Ayuda	Manual
-------	--------

9.3 Principios de Diseño

Aquí escribiría cosas como

- Responsive
- Mobile First
- Consistencia
- Accesibilidad
- Componentes reutilizables
- Navegación uniforme

10. Seguridad

La seguridad constituye uno de los pilares fundamentales del sistema Sazón Caribeño. Todas las operaciones críticas son protegidas mediante mecanismos de autenticación, autorización, validación de reglas de negocio y trazabilidad de las acciones realizadas por los usuarios.

La estrategia de seguridad del sistema se basa en múltiples capas de protección, reduciendo la posibilidad de accesos no autorizados y garantizando la integridad de la información almacenada.

10.1 Autenticación

10.2 Autorización (RBAC)

10.3 Heartbeat

10.4 Auditoría

10.5 Integridad

10.6 Contraseñas

10.7 Validaciones

10.8 Mejoras futuras

11. API REST

La comunicación entre el frontend y el backend de Sazón Caribeño se realiza mediante una API REST desarrollada con FastAPI. Esta interfaz constituye el punto central de acceso a las funcionalidades del sistema y permite desacoplar la lógica de negocio de la interfaz de usuario.

Cada recurso del sistema es expuesto mediante endpoints organizados por módulos funcionales. Las solicitudes y respuestas utilizan el formato JSON, permitiendo una integración sencilla con clientes web, aplicaciones móviles o servicios externos.

La API implementa validaciones mediante esquemas Pydantic, control de acceso basado en roles (RBAC), autenticación del usuario y reglas de negocio antes de ejecutar cualquier operación sobre la base de datos.

Todas las operaciones protegidas requieren que el usuario se encuentre autenticado y cuente con los permisos necesarios para ejecutar la acción solicitada.

11.1 Convenciones de la API

Con el propósito de mantener una interfaz consistente, la API adopta las siguientes convenciones generales:

- Todas las solicitudes y respuestas utilizan el formato JSON.
- Los endpoints siguen una estructura organizada por recursos.

- Las respuestas exitosas utilizan códigos HTTP de la familia 2xx.
- Los errores de validación utilizan códigos HTTP apropiados de acuerdo con la naturaleza del problema.
- Todos los datos recibidos son validados mediante esquemas Pydantic antes de ser procesados.
- Las operaciones críticas aplican las reglas de negocio definidas en el sistema antes de modificar la información almacenada.
- Los endpoints protegidos requieren autenticación y autorización según el rol del usuario.
- Las operaciones administrativas quedan registradas mediante el Servicio de Auditoría.

Módulo	Método	Endpoint	Descripción	Roles
Usuarios	POST	/api/usuarios/	Registrar usuario	Administrador
Usuarios	GET	/api/usuarios/	Listar usuarios	Administrador
Usuarios	GET	/api/usuarios/yo	Obtener usuario autenticado	Todos
Usuarios	PUT	/api/usuarios/{id}/reset-password	Restablecer contraseña	Administrador
Turnos	POST	/api/turnos/iniciar/{id}	Iniciar turno	Vendedor
Turnos	POST	/api/turnos/heartbeat/{id}	Heartbeat	Vendedor
Turnos	GET	/api/turnos/	Consultar turnos	Administrador / Gerente
Turnos	PUT	/api/turnos/{id}	Modificar turno	Administrador / Gerente
Inventario	POST	/api/productos/	Registrar producto	Administrador / Gerente

Módulo	Método	Endpoint	Descripción	Roles
Inventario	GET	/api/productos/	Consultar productos	Administrador / Gerente
Inventario	POST	/api/inventario/retirar/	Retirar inventario	Administrador / Gerente
Menú	POST	/api/platillos/	Registrar platillo	Administrador / Gerente
Menú	GET	/api/platillos/	Consultar platillos	Todos
Menú	PUT	/api/platillos/{id}	Actualizar platillo	Administrador / Gerente
Menú	DELETE	/api/platillos/{id}	Eliminar platillo	Administrador
Ventas	POST	/api/ventas/	Registrar venta	Vendedor
Ventas	GET	/api/ventas/	Consultar ventas	Administrador / Gerente
Gastos	POST	/api/gastos/	Registrar gasto	Administrador / Gerente
Gastos	GET	/api/gastos/	Consultar gastos	Administrador / Gerente
Reportes	GET	/api/reportes/corte-diario	Corte diario	Administrador / Gerente
Nómina	GET	/api/nomina/calculo	Calcular nómina	Administrador / Gerente

12. Procesos del Negocio

Los procesos de negocio describen la secuencia de actividades que ejecuta el sistema para dar soporte a las operaciones del restaurante. Cada proceso integra reglas de negocio, validaciones de seguridad y operaciones sobre la base de datos con el propósito de garantizar la consistencia de la información y el correcto funcionamiento de la organización.

La descripción de estos procesos permite comprender la interacción existente entre los distintos módulos del sistema y constituye una referencia para futuras tareas de mantenimiento, evolución y pruebas funcionales.

Objetivo

Registrar una venta y actualizar automáticamente toda la información relacionada.

Actores

Vendedor

Precondiciones

Turno activo.

Usuario autenticado.

Inventario disponible.

Flujo principal

1 El vendedor selecciona los platillos.

2 El sistema calcula subtotal.

3 Calcula IVA.

4 Calcula total.

5 El usuario confirma.

6 VentaService valida reglas.

7 InventarioService verifica ingredientes.

8 Descuenta inventario.

9 Guarda venta.

10 Guarda detalle.

11 Registra auditoría.

12 Devuelve factura.

Postcondiciones

Venta registrada.

Inventario actualizado.

Auditoría creada.

Excepciones

No existe inventario.

No hay turno.

Usuario sin permisos.

12.2 Iniciar Turno

Mismo formato.

12.3 Finalizar Turno

Mismo formato.

12.4 Registrar Gasto

Mismo formato.

12.5 Retirar Inventario

Mismo formato.

12.6 Crear Platillo

Mismo formato.

12.7 Calcular Nómina

Mismo formato.

12.8 Generar Corte Diario

Mismo formato.

12.9 Administrar Usuarios

Mismo formato.

12.10 Restablecer Contraseña

Mismo formato.